Министерство образования Новосибирской области

ГБПОУ НСО «Новосибирский авиационный технический колледж имени Б.С. Галущака»

ОТЧЕТ

по лабораторной работе № 4

Циклы в языке Python

Вариант 2

Выполнил: студент группы ПР-25.106

Ф.И.О. Ушаков Ю.С.

Новосибирск 2026

СОДЕРЖАНИЕ

1 Цель и задачи работы. Теоретические сведения 3

2 Задание 1. Вывод чисел 4

Блок-схема задания 1 5

3 Задание 2. Вычисление логарифма 6

Блок-схема задания 2 7

Тестирование и сходимость ряда 8

4 Задание 3. Погашение кредита 9

Блок-схема задания 3 10

Тестирование задания 3 11

5 Ответы на контрольные вопросы. Вывод 12

1 ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Цель работы - научиться разрабатывать программы, реализующие циклические алгоритмы на языке Python.

Задачи работы: изучить циклы for и while; составить блок-схемы трех заданий второго варианта; написать программы; проверить результаты и сравнить скорость сходимости ряда при разных значениях аргумента.

Общие теоретические сведения

Цикл многократно выполняет один блок команд. Один проход по телу цикла называется итерацией. Команды тела цикла записываются с одинаковым отступом, после заголовка ставится двоеточие.

Цикл for перебирает элементы последовательности. Функция range(start, finish, step) задает начальное значение, границу и шаг. Конечная граница в последовательность не входит. Отрицательный шаг позволяет перебирать числа по убыванию.

Цикл while повторяет команды, пока условие истинно. Условие проверяется перед каждой итерацией. Если оно сразу ложно, тело цикла не выполнится ни разу. Внутри цикла нужно изменять данные, от которых зависит условие завершения.

Оператор break немедленно завершает ближайший цикл. Оператор continue пропускает оставшуюся часть текущей итерации. Блок else после цикла выполняется при завершении без break.

Каждое задание находится в отдельном файле. Для проверки нужно открыть файл в Visual Studio Code, запустить его и ввести числа в терминале. Вещественные числа вводятся через точку.

2 ЗАДАНИЕ 1

Условие: даны два целых числа A и B, A < B. Вывести по убыванию все целые числа между A и B, не включая границы, и количество N этих чисел.

Описание алгоритма

После ввода проверяется условие A < B. Счетчик count начинается с нуля. Цикл for перебирает числа от B - 1 до A + 1 с шагом -1. Каждое число выводится, счетчик увеличивается на единицу. После цикла выводится количество чисел.

Исходный код

```
a = int(input("Введите A: "))
b = int(input("Введите B: "))

if a >= b:
    print("Ошибка: A должно быть меньше B")
else:
    count = 0
    print("Числа между A и B:")
    for number in range(b - 1, a, -1):
        print(number, end=" ")
        count += 1
    print("\nКоличество:", count)
```

Тестирование и результаты

A; B	Числа или сообщение	N
2; 7	6, 5, 4, 3	4
-3; 2	1, 0, -1, -2	4
4; 5	Чисел нет	0
0; 1	Чисел нет	0
-5; -1	-2, -3, -4	3
5; 5	Ошибка: A должно быть меньше B	-
7; 2	Ошибка: A должно быть меньше B	-

При A = 2 и B = 7 программа выводит 6, 5, 4, 3. Числа 2 и 7 не выводятся. Если границы соседние, цикл не выполняется, количество равно нулю.

БЛОК-СХЕМА ЗАДАНИЯ 1

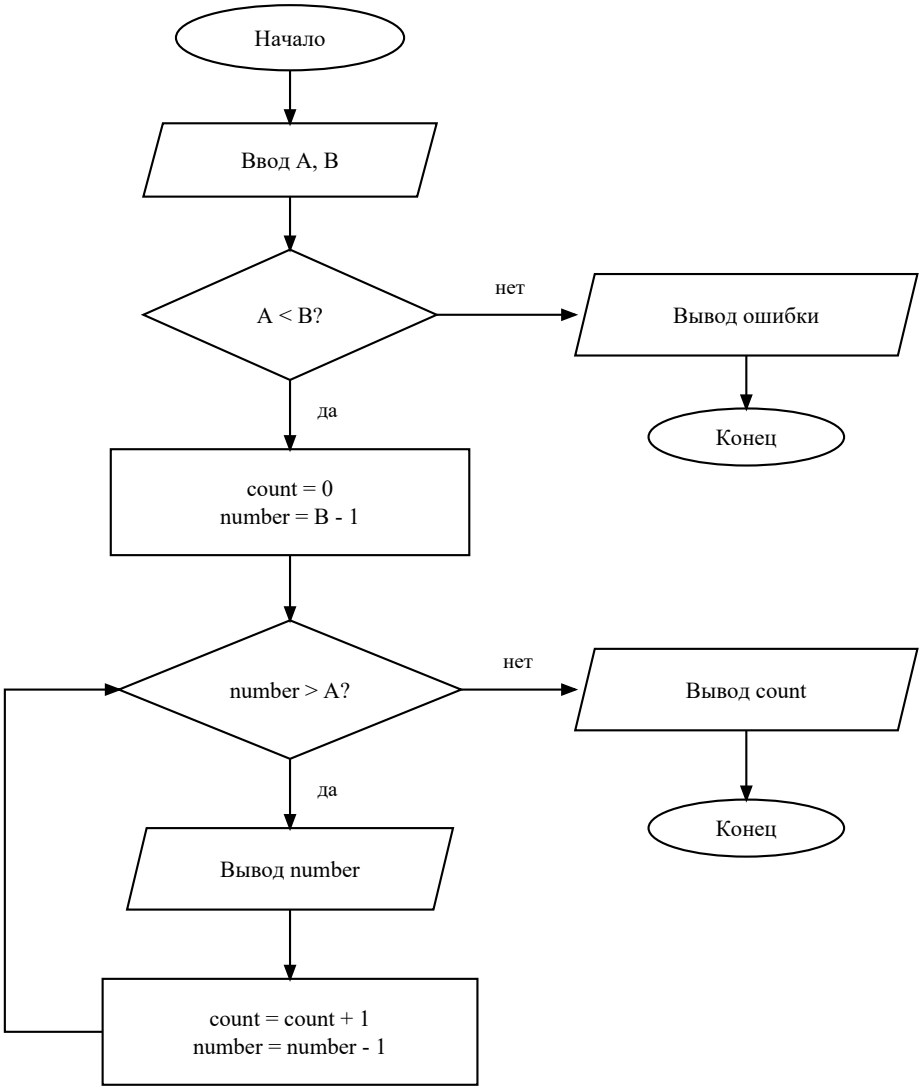


Рисунок 1 - Блок-схема вывода чисел по убыванию

Перебор number на схеме соответствует циклу for с range(B - 1, A, -1) в программе.

3 ЗАДАНИЕ 2

Условие: проверить равенство $\ln(1 - x) = -(x + x^2/2 + x^3/3 + ... + x^n/n + ...)$. Для заданных x и ε вычислить логарифм и сумму ряда, вывести число слагаемых и исследовать сходимость.

Описание алгоритма

Ряд сходится при $-1 \leq x < 1$. Вводятся x и положительная точность eps. Первое слагаемое term равно -x. В цикле слагаемое добавляется к total, счетчик n увеличивается, следующее слагаемое получается умножением предыдущего на $x \cdot n / (n + 1)$. Цикл продолжается, пока оценка остатка error больше eps. Число итераций ограничено одним миллионом; при недостижении точности выводится предупреждение.

Исходный код

```
from math import log1p

x = float(input("Введите x (-1 <= x < 1): "))
eps = float(input("Введите точность (например, 0.001): "))

if not -1 <= x < 1 or eps <= 0:
    print("Ошибка: нужны -1 <= x < 1 и положительная точность")
else:
    total = 0
    n = 0
    term = -x
    error = abs(term)
    if x > 0:
        error = error / (1 - x)

    while error > eps and n < 1000000:
        total += term
        n += 1
        # Следующее слагаемое: -x ** (n + 1) / (n + 1)
        term = term * x * n / (n + 1)
        error = abs(term)
        if x > 0:
            error = error / (1 - x)

    print("Сумма ряда:", total)
    print("ln(1 - x):", log1p(-x))
    print("Разница:", abs(total - log1p(-x)))
    print("Количество слагаемых:", n)
    if error > eps:
        print("За 1000000 итераций заданная точность не достигнута")
```

БЛОК-СХЕМА ЗАДАНИЯ 2

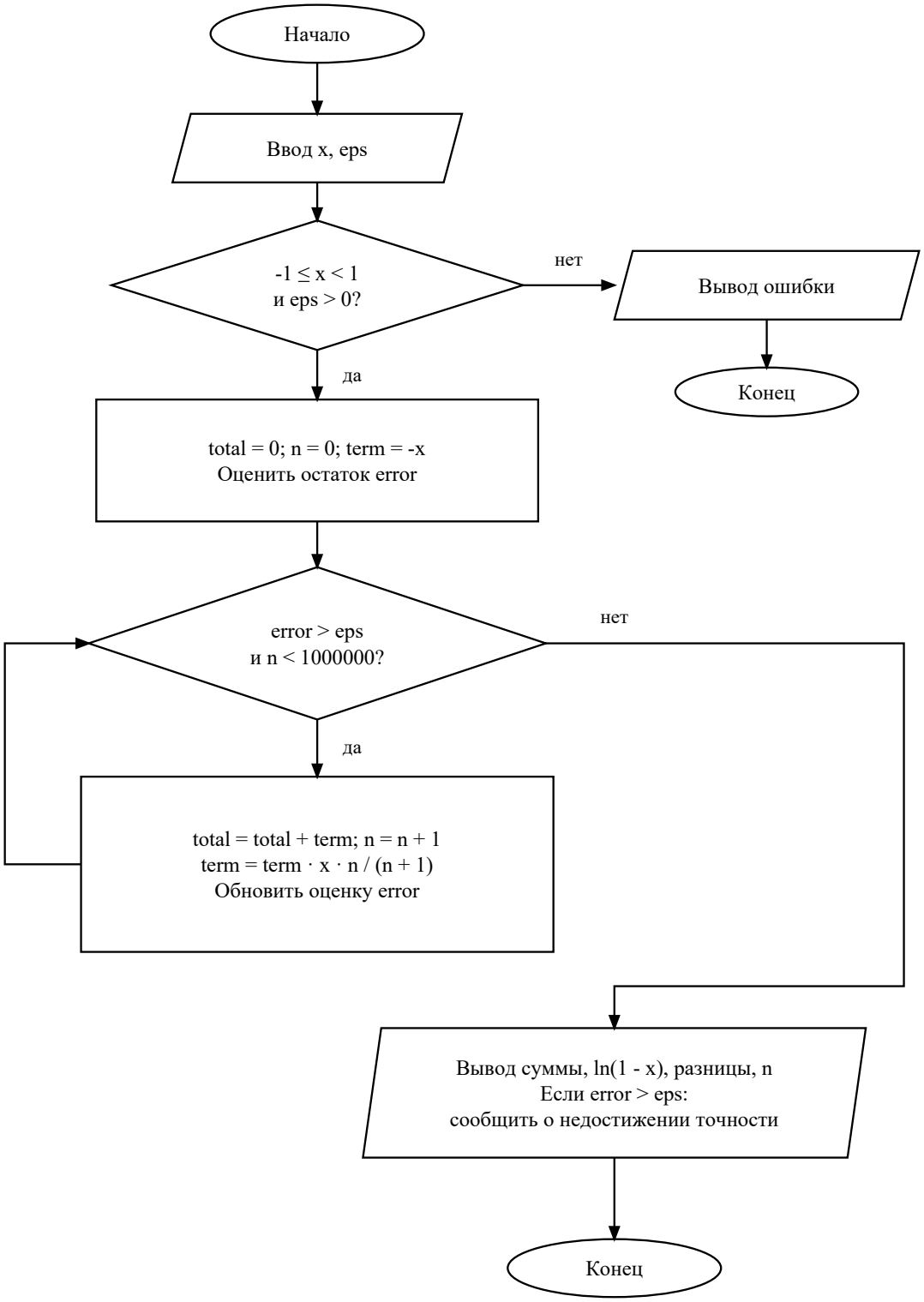


Рисунок 2 - Блок-схема вычисления логарифма через ряд

ТЕСТИРОВАНИЕ И СХОДИМОСТЬ РЯДА

Оценка остатка

Для $0 < x < 1$ все слагаемые одного знака. Модуль остатка не превосходит модуля следующего слагаемого, деленного на $1 - x$. Для $-1 \le x < 0$ ряд знакочередующийся: достаточно модуля следующего слагаемого. При $x = 0$ сумма сразу равна нулю. Именно эти оценки записываются в error.

Библиотечное значение вычисляется функцией log1p(-x), которая возвращает ln(1 - x). Оно используется для сравнения с суммой, а не для управления циклом. Оценка остатка относится к усечению ряда; небольшие ошибки округления вещественных чисел возможны.

Результаты запусков

x	eps	Сумма ряда	ln(1 - x)	n	Разница
0	0.001	0.00000000	-0.00000000	0	0.00000000
0.1	0.001	-0.10500000	-0.10536052	2	0.00036052
0.5	0.001	-0.69226190	-0.69314718	7	0.00088528
0.9	0.001	-2.30179625	-2.30258509	50	0.00078884
-0.5	0.001	0.40580357	0.40546511	7	0.00033846
-1	0.001	0.69364743	0.69314718	999	0.00050025
0.5	1e-06	-0.69314633	-0.69314718	16	0.00000085
0.99	1e-06	-4.60516927	-4.60517019	1133	0.00000092
-0.99	1e-06	0.68813414	0.68813464	720	0.00000050

При eps = 0.001 для x = 0.1 понадобилось 2 слагаемых, для x = 0.5 - 7, для x = 0.9 - 50. При приближении x к 1 сходимость замедляется. В точке x = -1 достигнута точность 0.001 за 999 слагаемых. Для x = 0.5 уменьшение eps до 0.000001 увеличило число слагаемых с 7 до 16.

Дополнительно проверены x = 1, x = -2 и неположительная точность: программа выводит ошибку. Для x = -1 и eps = 0.000000001 проверена остановка после миллиона итераций с предупреждением.

4 ЗАДАНИЕ 3

Условие: предприниматель взял кредит k рублей под p процентов годовых. Ежегодная прибыль равна r рублей. Определить, сможет ли он накопить сумму для погашения кредита и через сколько лет.

Принятая модель и алгоритм

До полного погашения платежей нет. В конце каждого года проценты прибавляются к долгу, а прибыль копится отдельно без процентов. Программа начинает с debt = k, savings = 0, years = 0. Пока накоплений недостаточно, вычисляются годовые проценты interest. Если прибыль не превышает проценты, разрыв уже не сможет уменьшиться, поэтому выполняется break. Иначе увеличиваются долг, накопления и число лет. При savings ≥ debt выводится срок погашения.

Исходный код

```
k = float(input("Введите размер кредита: "))
p = float(input("Введите годовой процент: "))
r = float(input("Введите прибыль за год: "))

if k <= 0 or p < 0 or r < 0:
    print("Ошибка: кредит должен быть положительным,",
          "процент и прибыль - неотрицательными")
else:
    debt = k
    savings = 0
    years = 0

    # Платежей до погашения нет, прибыль копится отдельно.
    while savings < debt:
        interest = debt * p / 100
        if r <= interest:
            break
        debt += interest
        savings += r
        years += 1

    if savings >= debt:
        print("Кредит можно погасить через", years, "лет")
        print("Накоплено:", round(savings, 2))
        print("Долг:", round(debt, 2))
    else:
        print("Накопить сумму для погашения кредита не получится")
```

БЛОК-СХЕМА ЗАДАНИЯ 3

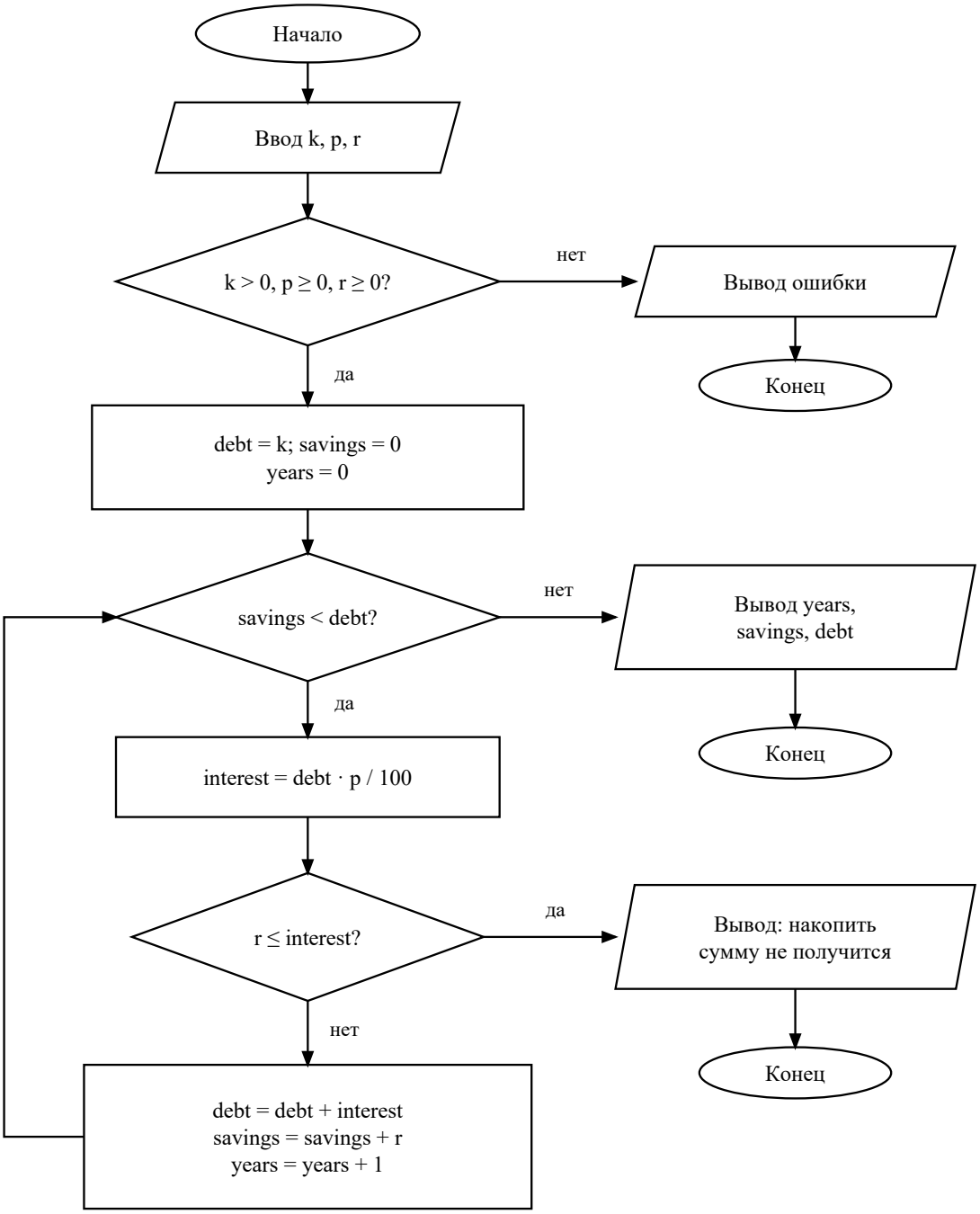


Рисунок 3 - Блок-схема проверки возможности погашения кредита

ТЕСТИРОВАНИЕ ЗАДАНИЯ 3

Для проверки использованы случаи погашения, невозможности погашения, нулевой ставки, нулевой прибыли и точного равенства накоплений долгу.

k; p; r	Ожидаемый срок в годах	Полученный результат
1000; 10; 300	6	6
1000; 10; 200	Погашение невозможно	Погашение невозможно
1000; 10; 100	Погашение невозможно	Погашение невозможно
1000; 0; 300	4	4
1000; 10; 1100	1	1
1000; 0; 0	Погашение невозможно	Погашение невозможно
1000; 10; 0	Погашение невозможно	Погашение невозможно
1000; 0; 1000	1	1
1000; 100; 2000	1	1

Проверка результата

При k = 1000, p = 10 и r = 300 через 5 лет накопится 1500 рублей, а долг составит 1610.51 рубля. Через 6 лет накопится 1800 рублей при долге 1771.56 рубля. Впервые накоплений достаточно на шестой год.

При k = 1000, p = 10 и r = 200 прибыль сначала превышает проценты, но в дальнейшем проценты растут. Если до погашения interest становится не меньше r, долг увеличивается не медленнее накоплений. Поэтому одних начальных условий $r > k \cdot p / 100$ недостаточно для положительного ответа.

Дополнительно проверены отрицательные значения и нулевой размер кредита: выводится ошибка. Всего для трех программ выполнено 34 проверки. Полученные результаты совпали с ожидаемыми.

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Для чего используются циклы?

Циклы нужны для повторения одних и тех же действий. Например, можно вывести последовательность чисел, накопить сумму или выполнить расчет по годам.

2. Какие виды циклов есть в Python?

В Python используются for и while. Цикл for перебирает последовательность, while выполняет команды, пока условие истинно.

3. Каков синтаксис while?

Заголовок записывается как while условие: . На следующих строках с отступом располагаются команды тела цикла. Условие проверяется перед каждой итерацией.

4. Каков синтаксис for?

Заголовок записывается как for переменная in последовательность: . Далее с отступом записывается тело. Например, for i in range(5): перебирает числа от 0 до 4.

5. Для чего используется break?

Оператор break немедленно завершает ближайший цикл. В третьем задании он прекращает расчет, когда становится ясно, что накопить нужную сумму не получится.

Общий вывод

Изучены циклы for и while. Выполнены три задания второго варианта: вывод чисел, вычисление логарифма через ряд и расчет возможности погашения кредита. Составлены блок-схемы, проведено тестирование. Сравнение показало, что число слагаемых зависит от аргумента и заданной точности.